

Renforcement Java – Codage et étude guidée d'un programme

Note : Les classes *Lecture* et *Ecriture* doivent être utilisées.

L'algorithme suivant reçoit en entrée une suite de caractères v, r, b correspondant à des boules vertes, rouges ou blanches. Son rôle consiste à trier ces boules :

- les vertes au début.
- les blanches au milieu,
- les rouges à la fin.

Les indices (r, s, t) , initialisés respectivement à $(0, 0, \text{nombreBoules} - 1)$, évoluent en vérifiant les propriétés suivantes pour tout i dans $[0.. \text{nombreBoules} - 1]$:

- $0 \leq i < r \implies \text{tableauBoules}[i] = \text{vert}$
- $r \leq i < s \implies \text{tableauBoules}[i] = \text{blanc}$
- $s \leq i \leq t \implies \text{tableauBoules}[i] \text{ non triée}$
- $t < i < \text{nombreBoules} \implies \text{tableauBoules}[i] = \text{rouge}$

Algorithme

```
1  final int nombreBoules = 10;
2  char[] tableauBoules;
3  ecrireString("suite des " + nombreBoules + " boules : ");
4  tableauBoules = lireTableauBoules();
5  int r = 0, s = 0, t = nombreBoules - 1;
6  tant que ( s != t ) faire
7      /* tableauBoules[s] est la boule à tester */
8      cas tableauBoules[s] dans
9          cas 'v' : echange(r, s, tableauBoules);
10                 ++r;
11                 ++s;
12          cas 'b' : ++s;
13          cas 'r' : echange(s, t, tableauBoules);
14                 --t;
15                 ++s;
16          autre  : ecrireString("erreur : s = " + s +
17                               ", boule = " + tableauBoules[s]);
18
19          arrêr;
20      fcas;
21 fait;
ecrireString("résultat du tri : ");
```

```

22 écrireTableauBoules (tableauBoules);
23 écrireStringln("");

```

Fonctions

```

1 char[] lireTableauBoules()
2 /* résultat : tableau contenant la suite des caractères lus
3  */
4 début
5     char[] tab = new char[nombreBoules];
6     pour(int i = 0; i < nombreBoules; ++i) faire
7         tab[i] = lireChar();
8     fait;
9     retourne tab;
10 fin

```

```

1 void écrireTableauBoules(char[] tab)
2 /* effet : le contenu de tab est affiché à l'écran
3  */
4 début
5     pour (int i = 0; i < nombreBoules; ++i) faire
6         écrireChar(tab[i]);
7     fait;
8 fin

```

```

1 void echange(int i, int j, char[] tab)
2 /* effet : échange tab[i] et tab[j]
3  */
4 début
5     char c = tab[i];
6     tab[i] = tab[j];
7     tab[j] = c;
8 fin

```

Questions

1. Coder "fidèlement" cet algorithme en Java où arrêt se traduit par `System.exit(0)`.
2. Exécuter le programme Java avec les jeux de données suivants (↵ est la touche Entrée) :
 - bvr rvv brbb↵ résultat ?
 - bvbvrrrvbb↵ résultat ?
 Quelle est votre conclusion ?
3. Pour visualiser le déroulement du programme, coder et inclure une fonction photo appelée dans l'itération après `fcas` ;

```
1 void photo(int r, int s, int t, char[] tab)
2 // effet : afficher les valeurs intermédiaires
3 //           des indices r, s, t et de tab
4 début
5     écrireStringln("r = " + r + " s = " + s + " t = " + t);
6     écrireTableauBoules(tab);
7     écrireStringln("");
8 fin
```

4. Corriger le programme Java, puis l'exécuter en suivant le déroulement pas à pas. Faire les tests avec les deux jeux de données :
 - bvbvrrrvvbb↵ résultat ?
 - bvbvrrbbvr↵ résultat ?Corriger les erreurs et réeffectuer les tests.
5. Le programme donné permet de trier 10 boules. Comment le généraliser pour trier un nombre quelconque positif de boules ?
6. Modifier la fonction lireTableauBoules afin d'accepter une suite de boules saisie au clavier sur plusieurs lignes et non plus nécessairement sur une seule ligne. Par exemple, avec la saisie sur trois lignes de : bvbv↵ rrb↵ bvr↵ on doit avoir le même résultat qu'avec la saisie de bvbvrrbbvr↵ sur une seule ligne.